

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №4
по дисциплине «Нейронные сети»
Тема: Распознавание рукописных символов

Студент гр. 1306

Шаврин А.П.

Преподаватель

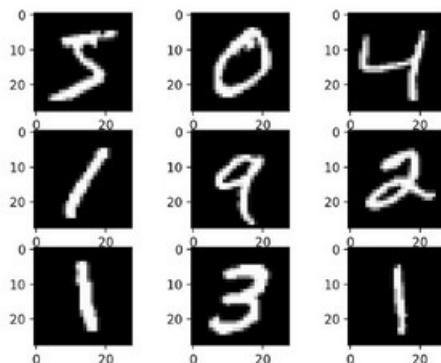
Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы.

Реализовать классификацию черно-белых изображений рукописных цифр (28x28) по 10 категориям (от 0 до 9).



Набор данных содержит 60,000 изображений для обучения и 10,000 изображений для тестирования.

Подробнее: <https://keras.io/datasets/> <http://yann.lecun.com/exdb/mnist/>

Задание.

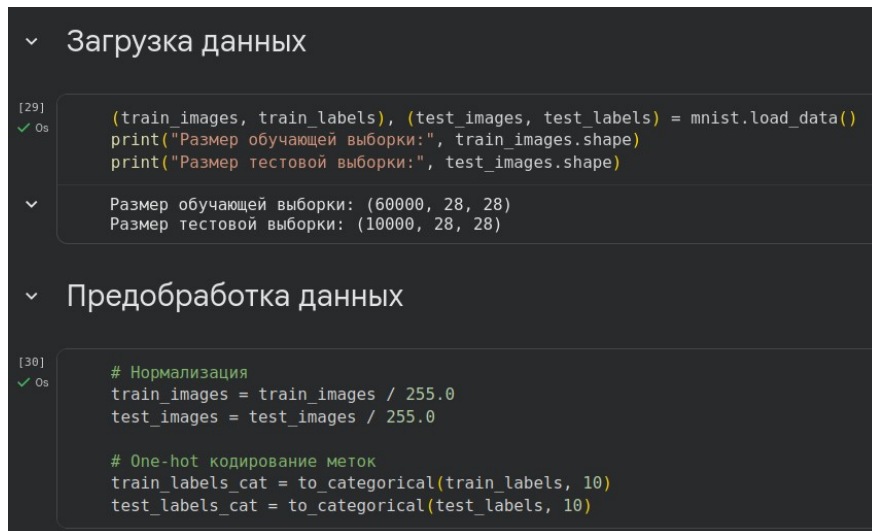
- Ознакомиться с представлением графических данных
- Ознакомиться с простейшим способом передачи графических данных
- нейронной сети
- Создать модель
- Настроить параметры обучения
- Написать функцию, позволяющая загружать изображение пользователя и
- классифицировать его

Выполнение работы.

Для работы использован встроенный в Keras набор данных MNIST, который содержит 60 000 изображений для обучения и 10 000 для тестирования. Загрузка выполнена с помощью `mnist.load_data()`.

Изображения представлены массивами чисел в диапазоне [0, 255]. Для приведения их к формату, удобному для обучения, выполнена нормировка. Метки преобразованы в one-hot представление с помощью `to_categorical`, что

необходимо для использования категориальной кросс-энтропии в качестве функции потерь. (рисунок 1).



```

▼ Загрузка данных

[29] ✓ 0s
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()
print("Размер обучающей выборки:", train_images.shape)
print("Размер тестовой выборки:", test_images.shape)

▼
Размер обучающей выборки: (60000, 28, 28)
Размер тестовой выборки: (10000, 28, 28)

▼ Предобработка данных

[30] ✓ 0s
# Нормализация
train_images = train_images / 255.0
test_images = test_images / 255.0

# One-hot кодирование меток
train_labels_cat = to_categorical(train_labels, 10)
test_labels_cat = to_categorical(test_labels, 10)

```

Рисунок 1. Чтение и предобработка данных.

Для классификации использована полносвязная нейронная сеть следующей архитектуры:

- Входной слой Flatten, преобразующий двумерное изображение 28×28 в одномерный вектор (784 признака).
- Скрытый слой из 256 нейронов с функцией активации ReLU.
- Выходной слой из 10 нейронов с функцией активации softmax для получения вероятностей принадлежности каждому классу.

Такая архитектура обеспечивает достаточную емкость для решения задачи MNIST и при правильном подборе гиперпараметров позволяет достичь точности выше 95%.

Для удобства проведения экспериментов написана функция, которая создает модель заданной архитектуры, затем компилирует ее с переданным оптимизатором, функцией потерь categorical_crossentropy и метрикой точности, а после обучает модель на обучающей выборке в течение 5 эпох с размером пакета 128 и оценивает точность на тестовой выборке, с последующим возвратом истории обучения и значением точности. Это позволило систематически исследовать различные оптимизаторы и их параметры без

дублирования кода. Функции генерации модели и обучения представлены на рисунке 2.

```

  ▾ Генерация модели

[31]
✓ Os
def create_model():
    model = Sequential([
        Flatten(input_shape=(28, 28)),
        Dense(256, activation='relu'),
        Dense(10, activation='softmax')
    ])
    return model

[32]
✓ Os
def train_and_evaluate(optimizer, epochs=5, batch_size=128):
    model = create_model()
    model.compile(optimizer=optimizer,
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

    history = model.fit(train_images, train_labels_cat,
                        epochs=epochs,
                        batch_size=batch_size,
                        validation_data=(test_images, test_labels_cat),
                        verbose=0)

    test_loss, test_acc = model.evaluate(test_images, test_labels_cat, verbose=0)
    return history, test_acc

```

Рисунок 2. Функции генерации модели и обучения.

1. Исследование влияния оптимизаторов и их параметров

Для исследования были выбраны три популярных оптимизатора: SGD, Adam и RMSprop. Каждый из них исследовался путем варьирования одного из ключевых параметров при фиксированных остальных (равных значениям по умолчанию). Такой подход позволяет оценить чувствительность алгоритма к конкретному параметру.

SGD (стохастический градиентный спуск)

Базовый метод оптимизации, обновляющий веса модели в направлении, противоположном градиенту функции потерь, вычисленному на небольшом пакете данных. Формула обновления:

$$w_{[t+1]} = w_t - \eta \nabla L(w_t)$$

где η - скорость обучения (learning rate), w_t - это вектор весов модели на текущем шаге (эпохе) t , L - функция потерь (loss function), $\nabla L(w_t)$ - градиент функции потерь по весам.

Модификация с моментом (momentum): Добавление момента позволяет ускорить сходимость и сгладить колебания. Обновление происходит с учетом предыдущего шага:

$$v_t = \mu v_{(t-1)} - \eta \nabla L(w_t)$$

$$w_{(t+1)} = w_t + v_t$$

Где параметр μ (momentum) определяет вклад предыдущего направления, а v_t - накопленная скорость на текущем шаге. Она хранит экспоненциально затухающее среднее предыдущих градиентов.

Исследовались два параметра:

- learning rate (определяет величину шага): 0.1, 0.01, 0.001, 0.0001.
- momentum (коэффициент инерции): 0.0, 0.5, 0.9, 0.99 (при фиксированном $lr=0.01$).

Результаты точности представлены на рисунке 3.

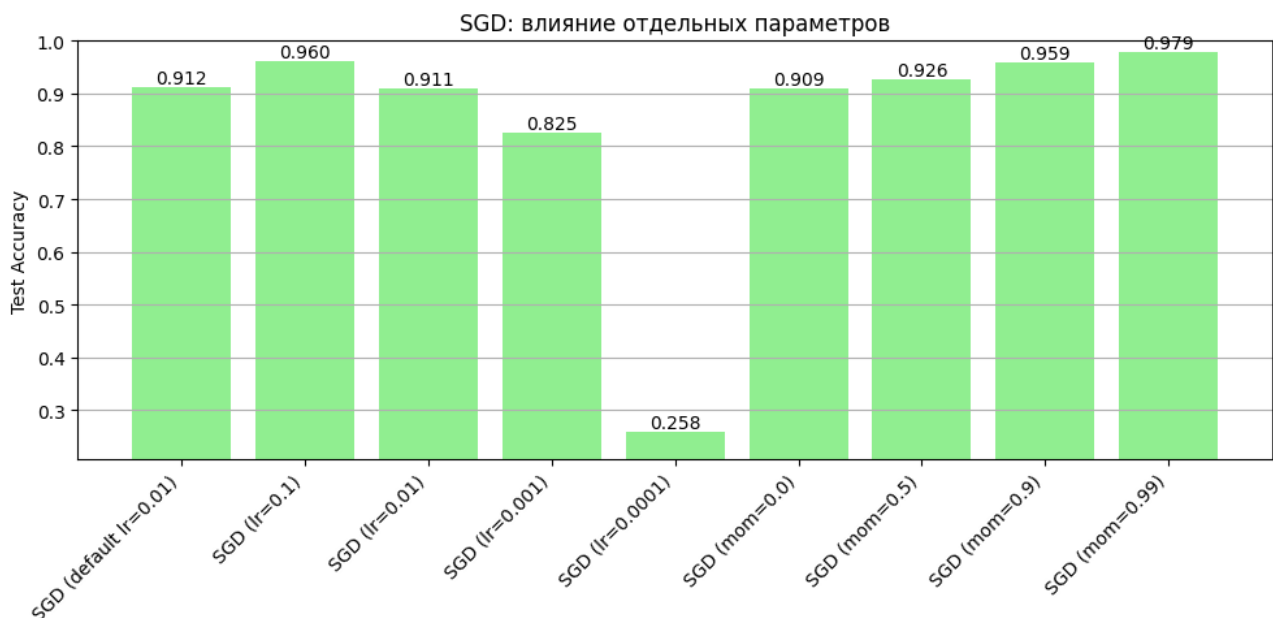


Рисунок 3. Точность модели с SGD при различных гиперпараметрах.

Скорость обучения 0.1 дает наилучший результат среди исследованных значений, но при этом шаг может быть слишком большим, что иногда приводит к расходимости (в паре с высоким моментом позже это подтвердилось).

Значения 0.01 и 0.001 дают постепенное снижение точности из-за недостаточной скорости обучения. При 0.0001 обучение практически не происходит (точность 25%).

Увеличение момента значительно улучшает сходимость, особенно $\mu=0.99$ дает почти 98% точности. Момент помогает накапливать инерцию и быстрее преодолевать локальные минимумы и пологие участки.

Adam

Алгоритм сочетает идеи момента и адаптивной подстройки скорости обучения для каждого параметра. Он хранит скользящие средние градиента (первый момент) и квадрата градиента (второй момент) и использует их для адаптивного шага. Параметры β_1 и β_2 управляют затуханием скользящих средних (обычно $\beta_1=0.9$, $\beta_2=0.999$).

Исследовались параметры:

- learning rate: 0.1, 0.01, 0.001, 0.0001.
- β_1 : 0.8, 0.9, 0.95, 0.99.
- β_2 : 0.9, 0.99, 0.999, 0.9999.

Результаты точности представлены на рисунке 4.

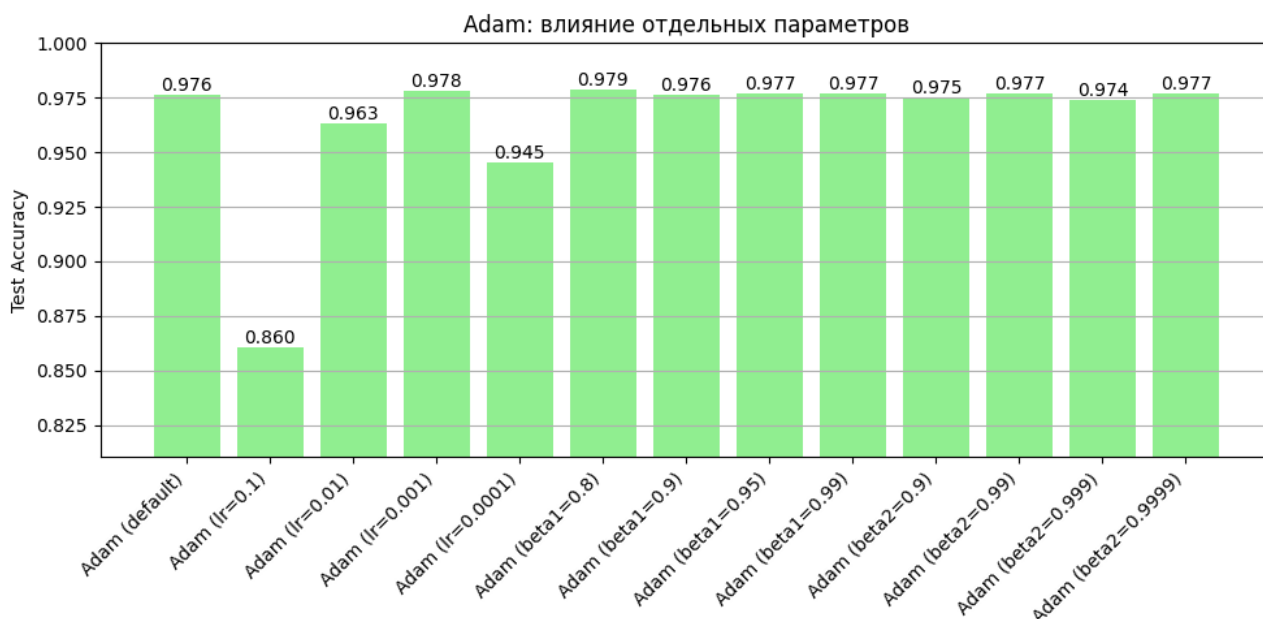


Рисунок 4. Точность модели с Adam при различных гиперпараметрах.

Лучшая скорость обучения - 0.001, что близко к значению по умолчанию. При 0.01 точность чуть ниже, при 0.1 - значительно ниже из-за неустойчивости, при 0.0001 - недостаточная скорость.

Параметр beta_1 слабо влияет на результат, все значения дают около 97.6–97.8%. Наибольшая точность при beta_1=0.8.

Параметр beta_2 также мало влияет, наилучший результат при beta_1=0.99 (0.9771).

Adam в целом устойчив и демонстрирует высокую точность в широком диапазоне параметров, что объясняется адаптивной природой алгоритма.

RMSprop

RMSprop (Root Mean Square Propagation) адаптирует скорость обучения для каждого параметра, деля ее на скользящее среднее квадратов градиентов. Это помогает бороться с затуханием градиента и ускоряет сходимость. Параметр rho определяет скорость затухания среднего. Модификация с моментом может включать момент аналогично SGD, что улучшает сходимость.

Исследовались:

- learning rate: 0.1, 0.01, 0.001, 0.0001.

- rho: 0.9, 0.95, 0.99.
- momentum: 0.0, 0.5, 0.9.

Результаты точности представлены на рисунке 5.

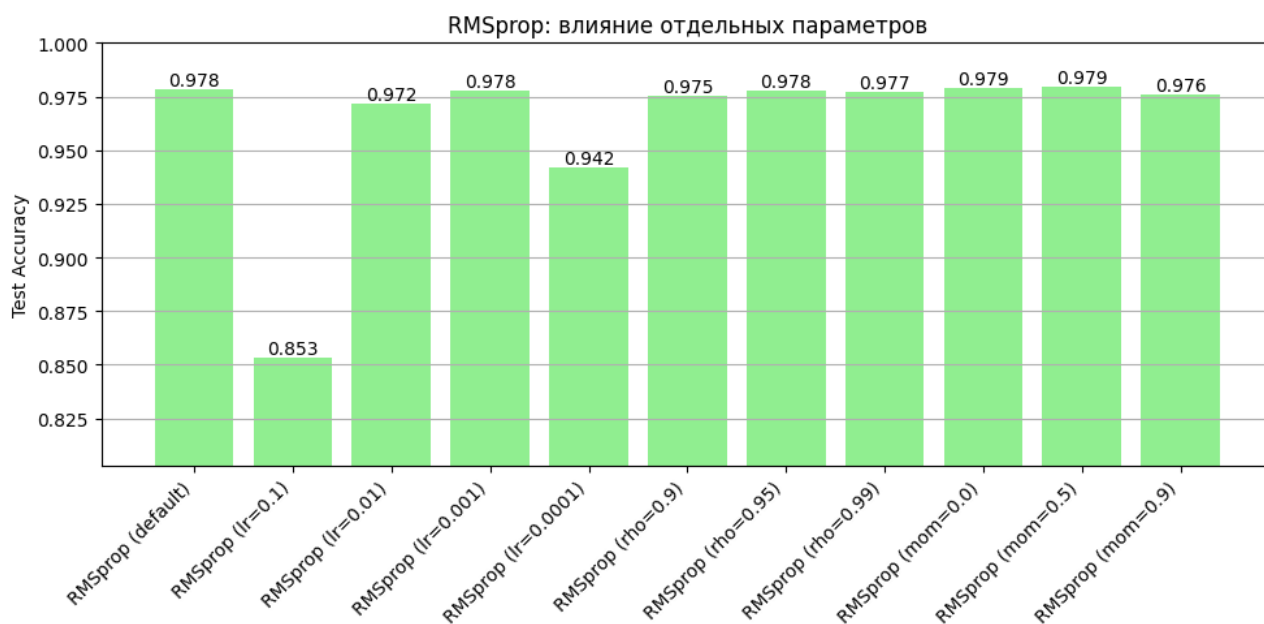


Рисунок 5. Точность модели с RMSprop при различных гиперпараметрах.

Наилучшая скорость обучения - 0.001 (как и для Adam).

Параметр rho оптимален при 0.95, что близко к стандартному 0.9

Добавление момента полезно: при $\mu=0.5$ точность достигает максимума (0.9793). При $\mu=0.9$ результат чуть ниже, возможно, из-за излишней инерции. RMSproof с моментом 0.5 показал лучший результат среди всех исследованных конфигураций.

2. Сравнение трех оптимизаторов с лучшими параметрами

На основе полученных данных для каждого оптимизатора были выбраны следующие комбинации:

- SGD: learning_rate=0.1, momentum=0.99
- Adam: learning_rate=0.001, beta_1=0.8, beta_2=0.99
- RMSprop: learning_rate=0.001, rho=0.95, momentum=0.5

Модели обучались с этими настройками, после чего был построен график точности на тестовых данных (рисунок 6) и графики потерь и точности на обучающей и валидационной выборках (рисунок 7-8).

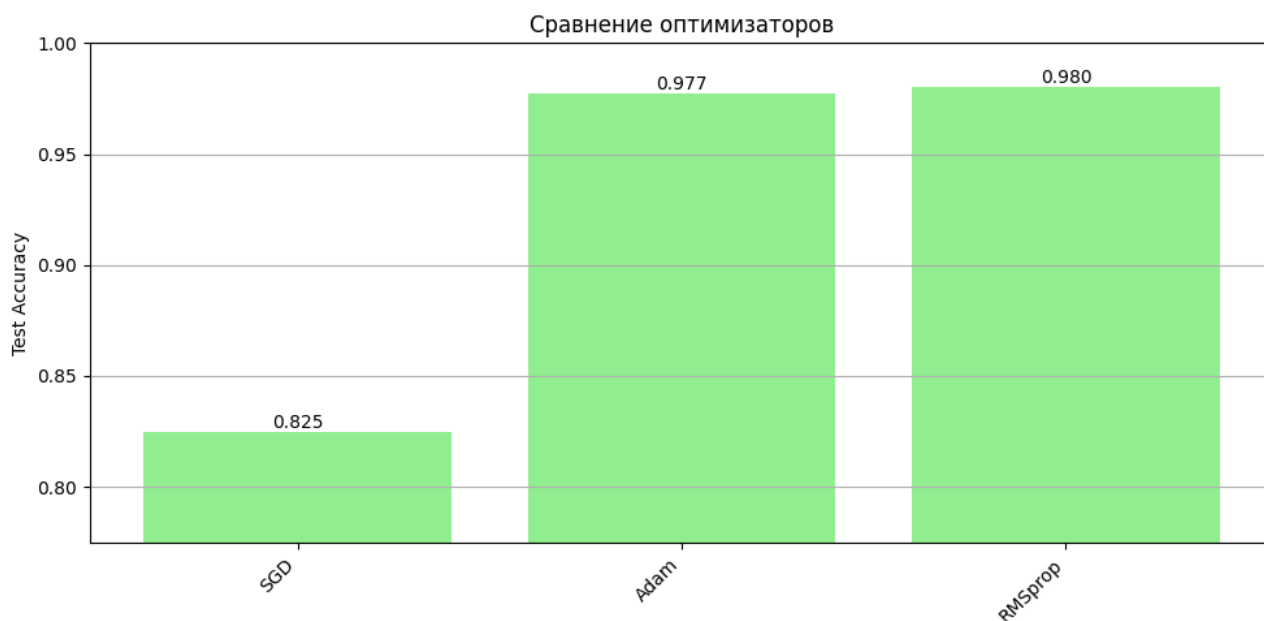


Рисунок 6. График точности для различных оптимизаторов.

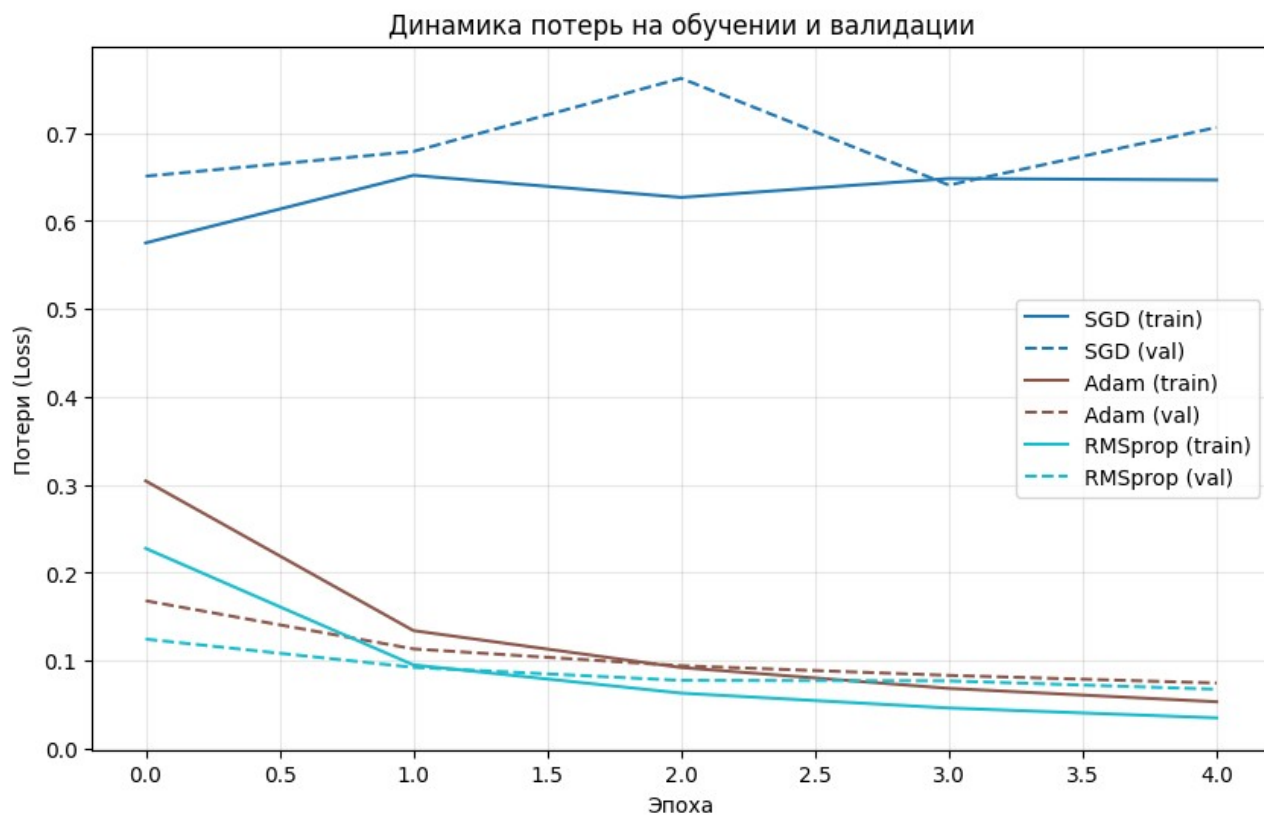


Рисунок 7. График потерь в ходе обучения для различных оптимизаторов.

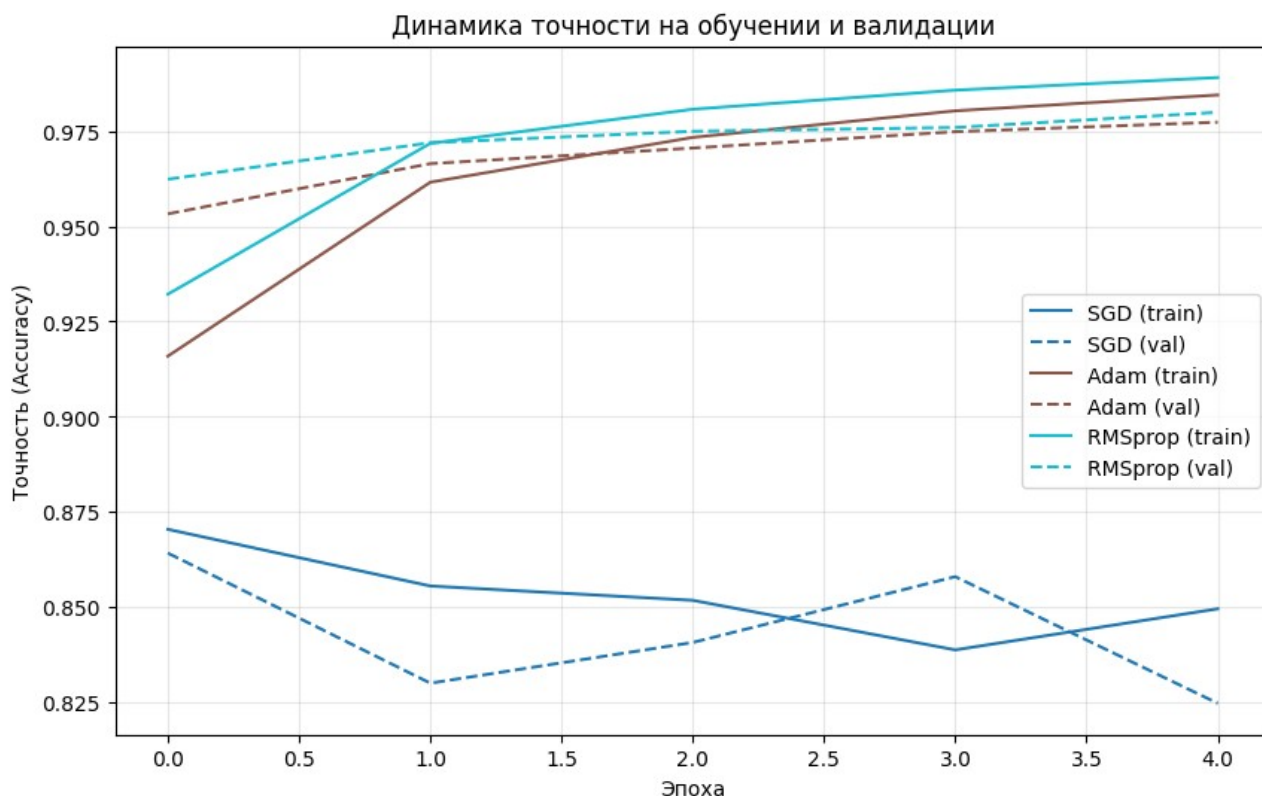


Рисунок 8. График точности в ходе обучения для различных оптимизаторов.

SGD с комбинацией большого шага (0.1) и высокого момента (0.99) показал нестабильность и низкую итоговую точность. Вероятно, такой большой шаг приводит к пропуску минимума, а высокий момент усугубляет колебания. Это контрастирует с отдельными экспериментами, где по отдельности эти параметры давали хорошие результаты, а их совместное использование оказалось неудачным.

Adam и RMSprop показали отличные результаты (около 98%). RMSprop с моментом 0.5 немного превзошел Adam. Оба алгоритма быстро сходятся уже после первой эпохи.

Лучший результат показал оптимизатор RMSprop с параметрами $lr=0.001$, $\rho=0.95$, $momentum=0.5$. Его точность на тесте достигла 98%, что превышает требуемый порог 95%

3. Реализация загрузки и классификации пользовательского изображения

Для демонстрации работы модели была написана функция

`load_and_preprocess_image`, которая:

- загружает изображение с диска,
- преобразует его в градации серого,
- масштабирует до размера 28×28 ,
- при необходимости инвертирует цвета (так как в датасете MNIST цифры изображены светлым на темном фоне),
- нормирует значения пикселей к диапазону $[0,1]$ и добавляет размерность пакета.

Функция `predict_user_image` принимает модель и путь к изображению, выполняет предсказание и возвращает предсказанный класс и уверенность. В качестве финальной модели использован RMSprop с параметрами ($lr=0.001$, $\rho=0.95$, $momentum=0.5$), обученный на 5 эпохах. На обучающей выборке точность достигла $\sim 99\%$.

После загрузки пользовательского файла (скриншот с цифрой 5) модель предсказала класс 5 с уверенностью 0.70. На графиках показаны исходное изображение и его обработанная версия 28×28 , что подтверждает корректность предобработки. На рисунке 9 представлен вывод исходного изображения с предсказанием и предобработанного изображения поданного на предсказание модели.

Исходное (предсказание: 5)



Обработанное (28x28)

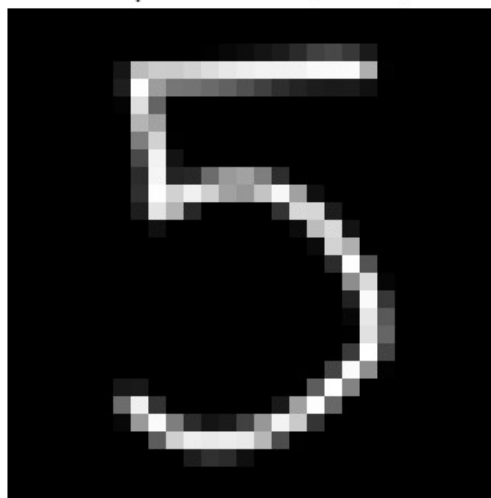


Рисунок 9. Обработка пользовательского изображения.

Выводы.

В ходе работы была реализована нейросетевая модель для классификации рукописных цифр MNIST, достигнута точность выше 95% на тестовых данных (98%). Проведено исследование трех оптимизаторов и их параметров, позволившее подобрать наилучшие настройки. Лучший результат (98%) показал оптимизатор RMSprop с параметрами $lr=0.001$, $\rho=0.95$, $momentum=0.5$. Разработана функция для загрузки и классификации пользовательских изображений, успешно протестированная на примере.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: lab4.ipynb

```
"""# Установка зависимостей
"""

!pip install pandas numpy matplotlib scikit-learn tensorflow

"""# Импорт"""

import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten, Dense
from tensorflow.keras import optimizers
import matplotlib.pyplot as plt
import numpy as np

"""# Загрузка данных"""

(train_images, train_labels), (test_images, test_labels) = mnist.load_data()
print("Размер обучающей выборки:", train_images.shape)
print("Размер тестовой выборки:", test_images.shape)

"""# Предобработка данных"""

# Нормализация
train_images = train_images / 255.0
test_images = test_images / 255.0

# One-hot кодирование меток
train_labels_cat = to_categorical(train_labels, 10)
test_labels_cat = to_categorical(test_labels, 10)

"""# Генерация модели"""

def create_model():
    model = Sequential([
        Flatten(input_shape=(28, 28)),
        Dense(256, activation='relu'),
        Dense(10, activation='softmax')
    ])
    return model

def train_and_evaluate(optimizer, epochs=5, batch_size=128):
    model = create_model()
    model.compile(optimizer=optimizer,
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])
```

```

history = model.fit(train_images, train_labels_cat,
                    epochs=epochs,
                    batch_size=batch_size,
                    validation_data=(test_images, test_labels_cat),
                    verbose=0)

test_loss, test_acc = model.evaluate(test_images, test_labels_cat, verbose=0)
return history, test_acc

"""# Исследование различных параметров оптимизаторов"""

def plot_optimizer_accuracy(optimizer_results, title):
    plt.figure(figsize=(10,5))
    bars = plt.bar(optimizer_results.keys(), optimizer_results.values(), color='lightgreen')
    plt.xticks(rotation=45, ha='right')
    plt.ylabel("Test Accuracy")
    plt.title(title)
    min_val = min(optimizer_results.values())
    max_val = 1.0
    delta = 0.05
    plt.ylim(min_val-delta, 1.0)
    plt.grid(axis='y')
    for bar in bars:
        height = bar.get_height()
        plt.text(bar.get_x() + bar.get_width()/2., height,
                 f'{height:.3f}', ha='center', va='bottom')
    plt.tight_layout()
    plt.show()

"""## SGD"""

sgd_results = {}
sgd_histories = {}

# Базовый SGD (lr=0.01)
base_sgd = optimizers.SGD(learning_rate=0.01)
hist_base, acc_base = train_and_evaluate(base_sgd)
sgd_results['SGD (default lr=0.01)'] = acc_base
sgd_histories['SGD (default lr=0.01)'] = hist_base
print(f"\nSGD (default lr=0.01): {acc_base:.4f}")

# 1. Варьируем learning rate
lr_list = [0.1, 0.01, 0.001, 0.0001]
for lr in lr_list:
    name = f'SGD (lr={lr})'
    opt = optimizers.SGD(learning_rate=lr)
    hist, acc = train_and_evaluate(opt)
    sgd_results[name] = acc
    sgd_histories[name] = hist
    print(f"{name}: {acc:.4f}")

```

```

# 2. Варьируем momentum (при фиксированном lr=0.01)
mom_list = [0.0, 0.5, 0.9, 0.99]
for mom in mom_list:
    name = f'SGD (mom={mom})'
    opt = optimizers.SGD(learning_rate=0.01, momentum=mom)
    hist, acc = train_and_evaluate(opt)
    sgd_results[name] = acc
    sgd_histories[name] = hist
    print(f'{name}: {acc:.4f}')

plot_optimizer_accuracy(sgd_results, "SGD: влияние отдельных параметров")

"""## Adam

adam_results = {}
adam_histories = {}

# Базовый Adam
base_adam = optimizers.Adam()
hist_base, acc_base = train_and_evaluate(base_adam)
adam_results['Adam (default)'] = acc_base
adam_histories['Adam (default)'] = hist_base
print(f'\nAdam (default): {acc_base:.4f}')

# 1. Варьируем learning rate
lr_list = [0.1, 0.01, 0.001, 0.0001]
for lr in lr_list:
    name = f'Adam (lr={lr})'
    opt = optimizers.Adam(learning_rate=lr)
    hist, acc = train_and_evaluate(opt)
    adam_results[name] = acc
    adam_histories[name] = hist
    print(f'{name}: {acc:.4f}')

# 2. Варьируем beta_1
beta1_list = [0.8, 0.9, 0.95, 0.99]
for b1 in beta1_list:
    name = f'Adam (beta1={b1})'
    opt = optimizers.Adam(beta_1=b1)
    hist, acc = train_and_evaluate(opt)
    adam_results[name] = acc
    adam_histories[name] = hist
    print(f'{name}: {acc:.4f}')

# 3. Варьируем beta_2
beta2_list = [0.9, 0.99, 0.999, 0.9999]
for b2 in beta2_list:
    name = f'Adam (beta2={b2})'
    opt = optimizers.Adam(beta_2=b2)
    hist, acc = train_and_evaluate(opt)

```

```

adam_results[name] = acc
adam_histories[name] = hist
print(f"{name}: {acc:.4f}")

plot_optimizer_accuracy(adam_results, "Adam: влияние отдельных параметров")

"""## RMSprop"""

rms_results = {}
rms_histories = {}

# Базовый RMSprop (lr=0.001)
base_rms = optimizers.RMSprop()
hist_base, acc_base = train_and_evaluate(base_rms)
rms_results['RMSprop (default)'] = acc_base
rms_histories['RMSprop (default)'] = hist_base
print(f"\nRMSprop (default): {acc_base:.4f}")

# 1. Варьируем learning rate
lr_list = [0.1, 0.01, 0.001, 0.0001]
for lr in lr_list:
    name = f'RMSprop (lr={lr})'
    opt = optimizers.RMSprop(learning_rate=lr)
    hist, acc = train_and_evaluate(opt)
    rms_results[name] = acc
    rms_histories[name] = hist
    print(f"{name}: {acc:.4f}")

# 2. Варьируем rho
rho_list = [0.9, 0.95, 0.99]
for rho in rho_list:
    name = f'RMSprop (rho={rho})'
    opt = optimizers.RMSprop(rho=rho)
    hist, acc = train_and_evaluate(opt)
    rms_results[name] = acc
    rms_histories[name] = hist
    print(f"{name}: {acc:.4f}")

# 3. Варьируем momentum
mom_list = [0.0, 0.5, 0.9]
for mom in mom_list:
    name = f'RMSprop (mom={mom})'
    opt = optimizers.RMSprop(momentum=mom)
    hist, acc = train_and_evaluate(opt)
    rms_results[name] = acc
    rms_histories[name] = hist
    print(f"{name}: {acc:.4f}")

plot_optimizer_accuracy(rms_results, "RMSprop: влияние отдельных параметров")

"""## Исследование различных оптимизаторов"""

```



```

def plot_loss(histories, names=None, figsize=(10, 6)):
    plt.figure(figsize=figsize)

    if names is None:
        names = list(histories.keys())

    colors = plt.cm.tab10(np.linspace(0, 1, len(names)))

    for idx, name in enumerate(names):
        hist = histories[name]
        color = colors[idx]
        plt.plot(hist.history['loss'], label=f'{name} (train)', color=color)
        plt.plot(hist.history['val_loss'], '--', label=f'{name} (val)', color=color)

    plt.xlabel('Эпоха')
    plt.ylabel('Потери (Loss)')
    plt.title('Динамика потерь на обучении и валидации')
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.show()

def plot_accuracy(histories, names=None, figsize=(10, 6)):
    plt.figure(figsize=figsize)

    if names is None:
        names = list(histories.keys())

    colors = plt.cm.tab10(np.linspace(0, 1, len(names)))

    for idx, name in enumerate(names):
        hist = histories[name]
        color = colors[idx]
        plt.plot(hist.history['accuracy'], label=f'{name} (train)', color=color)
        plt.plot(hist.history['val_accuracy'], '--', label=f'{name} (val)', color=color)

    plt.xlabel('Эпоха')
    plt.ylabel('Точность (Accuracy)')
    plt.title('Динамика точности на обучении и валидации')
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.show()

optimizers_list = {
    'SGD': optimizers.SGD(learning_rate=0.1, momentum=0.99),
    'Adam': optimizers.Adam(learning_rate=0.001, beta_1=0.8),
    'RMSprop': optimizers.RMSprop(learning_rate=0.001, rho=0.95, momentum=0.5),
}

results = {}
histories = {}

for name, opt in optimizers_list.items():

```

```

print(f"Обучение с {name}")
hist, acc = train_and_evaluate(opt)
results[name] = acc
histories[name] = hist
print(f"Точность на тесте: {acc:.4f}\n")

plot_optimizer_accuracy(results, "Сравнение оптимизаторов")

plot_loss(histories, names=list(histories.keys()))
plot_accuracy(histories, names=list(histories.keys()))

"""# Обработка пользовательского изображения"""

from google.colab import files
from PIL import Image
import io

def load_and_preprocess_image(image_path, invert=True):
    """Загружает изображение, подготавливает его для модели MNIST."""
    img = Image.open(image_path).convert('L') # в градации серого
    img = img.resize((28, 28))
    img_array = np.array(img, dtype=np.float32)

    if invert:
        img_array = 255 - img_array

    img_array = img_array / 255.0

    img_array = np.expand_dims(img_array, axis=0)
    return img_array

def predict_user_image(model, image_path, invert):
    processed = load_and_preprocess_image(image_path, invert=invert)
    prediction = model.predict(processed)
    predicted_class = np.argmax(prediction)
    confidence = np.max(prediction)
    return predicted_class, confidence

final_model = create_model()
final_optimizer = optimizers.RMSprop(learning_rate=0.001, rho=0.95, momentum=0.5)
final_model.compile(optimizer=final_optimizer,
                    loss='categorical_crossentropy',
                    metrics=['accuracy'])
final_model.fit(train_images, train_labels_cat, epochs=5, batch_size=128, verbose=1)

uploaded = files.upload()
for fn in uploaded.keys():
    print(f'Загружен файл: {fn}')
    pred, conf = predict_user_image(final_model, fn, invert=True)
    print(f'Предсказанная цифра: {pred}, уверенность: {conf:.2f}')
    # Отображение исходного изображения
    img = Image.open(fn)

```

```
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(img, cmap='gray')
plt.title(f'Исходное (предсказание: {pred})')
plt.axis('off')

# Отображение обработанного изображения (28x28)
plt.subplot(1, 2, 2)
plt.imshow(processed.squeeze(), cmap='gray')
plt.title('Обработанное (28x28)')
plt.axis('off')
plt.show()
```